

Otulia Web: Deep Dive - Chapter 1: Introduction & Architecture

1.1 The Vision of Otulia

Otulia is not just a marketplace; it is a digital sanctuary for luxury assets. The platform is designed to bridge the gap between high-net-worth individuals and elite dealers of Cars, Yachts, Estates, and Bikes. Every line of code in this project is optimized to provide a "White Glove" experience, focusing on high-resolution media, secure transactions, and a seamless interface.

1.2 The Monorepo Architecture

The project utilizes a monorepo structure. This means the frontend (`client`) and the backend (`server`) live in the same repository but remain decoupled in their logic.

1.2.1 Directory Map

- `/client` : The React application. It handles everything the user sees.
 - `/server` : The Node.js/Express API. It handles data, security, and integration with 3rd party services like Cloudinary and Stripe.
 - **Root Files:**
 - `package.json` : Manages orchestration scripts to build both halves of the app simultaneously.
 - `.gitignore` : Ensures secrets (`.env`) and heavy folders (`node_modules`) are never committed to version control.
-

1.3 The Tech Stack Deep-Dive

1.3.1 Frontend: React 19 & Vite

We use **React 19**, the latest version of the world's most popular UI library.

- **Why React?** It allows us to build a Single Page Application (SPA). When a user clicks "Shop," the page doesn't reload; the UI simply "swaps" components, making the site feel like a high-end mobile app.
- **Why Vite?** Older tools like Create React App are slow. Vite uses "Native ESM" to provide near-instant development starts and lightning-fast builds.

1.3.2 Styling: Tailwind CSS 4

The luxury aesthetic is achieved using **Tailwind CSS 4**.

- Instead of writing thousands of lines of custom CSS, we use utility classes directly in the HTML. This ensures the design is consistent across the entire site.
- **Premium Touches:** We use the `Montserrat` and `Playfair Display` fonts to evoke a sense of elegance and authority.

1.3.3 Backend: Node.js & Express 5

The backend is built on **Node.js** with **Express 5**.

- Express is minimal and unopinionated, allowing us to build custom routes for complex luxury data structures.

- It acts as the "Gatekeeper," verifying every user request via JWT (JSON Web Tokens) before allowing access to sensitive data like dealer contact info or admin dashboards.

1.3.4 Database: MongoDB & Mongoose

We use a **NoSQL** database (MongoDB) because luxury assets are complex.

- A "Car" has horsepower and engine type.
 - A "House" has bedrooms and square footage.
 - MongoDB allows us to store these different "shapes" of data in one place without the rigid constraints of a traditional SQL table.
-

1.4 Detailed File Purpose (Frontend)

1.4.1 client/src/main.jsx

The "Heartbeat" of the application.

```
// This is the entry point where React meets the Browser.
createRoot(document.getElementById('root')).render(
  <StrictMode>
    { /* Provides SEO capabilities to the whole app */ }
    <HelmetProvider>
      { /* Enables client-side navigation without page refreshes */ }
      <BrowserRouter>
        { /* Handles Google Sign-In across the entire app */ }
        <GoogleOAuthProvider clientId={...}>
          { /* Manages Login/Logout state globally */ }
          <AuthProvider>
            <App />
          </AuthProvider>
        </GoogleOAuthProvider>
      </BrowserRouter>
    </HelmetProvider>
  </StrictMode>,
)
```

1.4.2 client/src/App.jsx

The "Switchboard" of the application. It maps every URL (like `/shop` or `/profile`) to a specific Page component. It also handles logic for showing or hiding the **Footer** depending on the page (e.g., hiding it on the Admin Dashboard).

1.4.3 client/src/contexts/AuthContext.jsx

The "Memory" of the application.

- It remembers if a user is logged in.
 - It stores the **JWT Token**.
 - It provides functions like `login()`, `signup()`, and `logout()` to any component that needs them.
-

1.5 Detailed File Purpose (Backend)

1.5.1 server/index.js

The "Engine Room".

- It imports all the routes (auth, assets, inventory).
- It connects to the database using `connectDB()`.
- It sets up **CORS** (Cross-Origin Resource Sharing) to allow your frontend to talk to your backend securely.
- **Crucial Order:** It checks for the `/sitemap.xml` FIRST to ensure Google's bots get the data they need before anything else.

1.5.2 server/db.js

The "Safe" where the data lives.

```
const connectDB = async () => {
  try {
    // Uses the MONGO_URI from your .env file
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDb connected")
  } catch (error) {
    // If the database is down, the server stops immediately
    console.error("MongoDB error:", error.message);
    process.exit(1);
  }
};
```

1.6 The Life of a Request (Architecture Flow)

To understand how Otulia works, let's follow what happens when a user views a **Ferrari**:

1. **User Action:** User clicks on a Car Card.
2. **Frontend Route:** `App.jsx` detects the URL change to `/asset/car/123`.
3. **Component Load:** `Asset.jsx` loads and calls the backend API at `/api/assets/car/123`.
4. **Backend Route:** `server/index.js` sends the request to `assets.routes.js`.
5. **Database Query:** The `CarAsset` model queries MongoDB for ID `123`.
6. **Response:** The server sends the Ferrari data back as a JSON object.
7. **Render:** React receives the JSON, updates its state, and the Ferrari images appear on the screen instantly.

1.7 Security & Environment

Architecture is nothing without security.

- **.env Files:** We store API keys (Google, Cloudinary, Stripe) in these files. They are never uploaded to GitHub.
 - **JWT:** When a user logs in, they get a digital "Passport" (the token). For every action (like listing a car), they must show this passport to the server.
 - **Input Sanitization:** All data sent to MongoDB is cleaned via Mongoose schemas to prevent "Injection" attacks.
-

1.8 Development vs Production

The architecture is designed to scale.

- **Development:** You run `npm run dev` in the client and `node --watch index.js` in the server. This allows for "Hot Reloading"—you see your changes the moment you save a file.
- **Production:** You run `npm run build`. Vite shrinks and "obfuscates" your React code into tiny, fast files. The Node server then serves these files directly from the `client/dist` folder for maximum performance on Hostinger.